DSA0120-OBJECT OEIETED PROGRAMMING WITH C++
ASSIGNMENT 3 C01

JITHESH C (192524320)

Q1:

Q1 **Print a hollow diamond pattern**

Use nested loops to print a diamond of five rows in the upper half and four rows in the lower half.

```
*
* *
* *
* *
* *
* *
* *
* *
*
```

ANS:

Your Code

```
#include <iostream>
using namespace std;
int main() {
    int n = 5;
    for (int i = 1; i <= 2 * n - 1; i++) {
        int rows = (i <= n) ? i : 2 * n - i;
        for (int j = 1; j <= n - rows; j++) cout << " ";
        for (int j = 1; j <= 2 * rows - 1; j++) {
            if (j == 1 || j == 2 * rows - 1) cout << "*";
            else cout << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Submitted

Input

stdin input

Output

```
    *
   * *
  *   *
 *     *
*       *
 *     *
  *   *
   * *
    *
```

Q2:

Q2 Design a class to calculate parking charges 5 marks

Create a Parking class. Charge ₹30 for the first hour and ₹20 for every additional hour. Any fraction of an hour counts as a full hour.
Example: 2 hours and 15 minutes → ₹70.
TC_01:2 15->Parking charge: Rs. 70.TC_02:0 30->Parking charge: Rs. 30.TC_03:2 60->Invalid duration

ANS:

Your Code

```cpp
#include <iostream>
using namespace std;

class Parking {
private:
    int hours;
    int minutes;

public:
    void readInput(int h, int m) {
        hours = h;
        minutes = m;
    }

    bool isValid() {
        return (minutes >= 0 && minutes < 60);
    }

    int calculateCharge() {
        int totalHours = hours;
        if (minutes > 0) {
            totalHours++;
        }
        if (totalHours <= 0) {
            return 0;
        }
        int charge = 30;
        if (totalHours > 1) {
            charge += (totalHours - 1) * 20;
        }
        return charge;
```

Input

stdin input

Output

```
Parking charge: Rs. 30
```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

5.00 / 5 marks

Q3:

Q3 Print a number pattern with consecutive values 5 marks

Print the following pattern for four rows.

1
2 3
4 5 6
7 8 9 10

ANS:

Your Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int rows = 4;
    int count = 1;
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            cout << count;
            if (j < i) {
                cout << " ";
            }
            count++;
        }
        cout << endl;
    }
    return 0;
}
```

Submitted

Input

stdin input

Output

```
1
2 3
4 5 6
7 8 9 10
```

Q4:

Q4 **Check whether a number is a spy number** 5 marks

A number is a spy number if the sum of its digits equals the product of its digits.
Example: 1124 → Sum = 8, product = 8.
TC_01:1124->Spy number.TC_02:124->Not a spy number.TC_03:-123->Invalid input

ANS:

```cpp
#include <iostream>

int main() {
    int n;
    if (!(std::cin >> n)) return 0;

    if (n < 0) {
        std::cout << "Invalid input" << std::endl;
        return 0;
    }

    int sum = 0;
    int product = 1;
    int temp = n;


    if (temp == 0) {
        sum = 0;
        product = 0;
    } else {
        while (temp > 0) {
            int digit = temp % 10;
            sum += digit;
            product *= digit;
            temp /= 10;
```

1124

Output

Spy number

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

5.00 / 5 marks

Q5:

Q5 Print a butterfly pattern — 5 marks

Print a butterfly pattern using nested loops.

```
* *
** **
*** ***
**** ****
**********
**** ****
*** ***
** **
* *
```

ANS:

Your Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {

        for (int j = 1; j <= i; j++) {
            cout << "*";
        }

        int spaces = 2 * (n - i);
        for (int j = 1; j <= spaces; j++) {
            cout << " ";
        }

        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << endl;
    }

    for (int i = n - 1; i >= 1; i--) {

        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        int spaces = 2 * (n - i);
```

Input

```
stdin input
```

Output

```
**        **
***      ***
****    ****
**********
****    ****
***      ***
```